4장 데이터 시각화 도구 맷플롯립

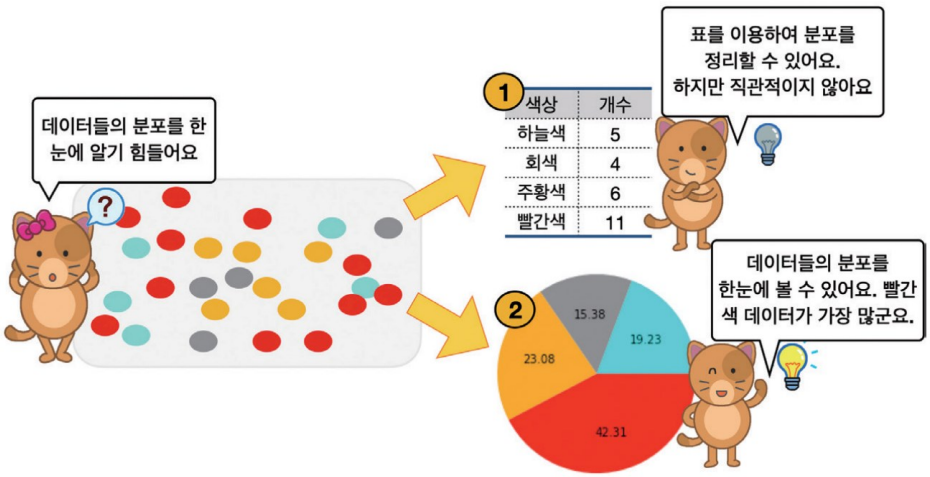
학습목표

- 효과적인 시각화 기법이 데이터 분석에서 매우 중요한 이유를 알아본다.
- 맷플롯립을 사용하여 다양한 함수를 효과적으로 시각화할 수 있다는 것을 이해한다.
- 필요한 만큼의 행과 열을 사용하여 화면 공간을 분할하여 사용하는 방법을 익힌다.
- 자료값의 분포를 나타내는 여러 가지 방법을 익힌다.
- 통계 데이터를 맷플롯립을 사용하여 효과적으로 시각화하는 방법을 익힌다.

4.1 데이터 과학과 효과적인 시각화의 필요성

- 이번 장에서는 **컴퓨터 화면에 시각적 이미지를 이용하여 데이터를 효과적으로 보여주는 방법**인 **데이터 시각화**data visualization에 대해 알아보자.
- 일반적으로 사람은 필요한 정보의 80% 가량을 시각 시스템을 통해서 받아들인다고 알려져 있다.
- 따라서, 효과적인 시각화는 사용자가 데이터를 분석하고 추론하는 데 도움이 된다. **사람들은 수치 데이터보다 시각적으로 보이는 그림을 더 직관적으로 이해**할 수 있기 때문이다.

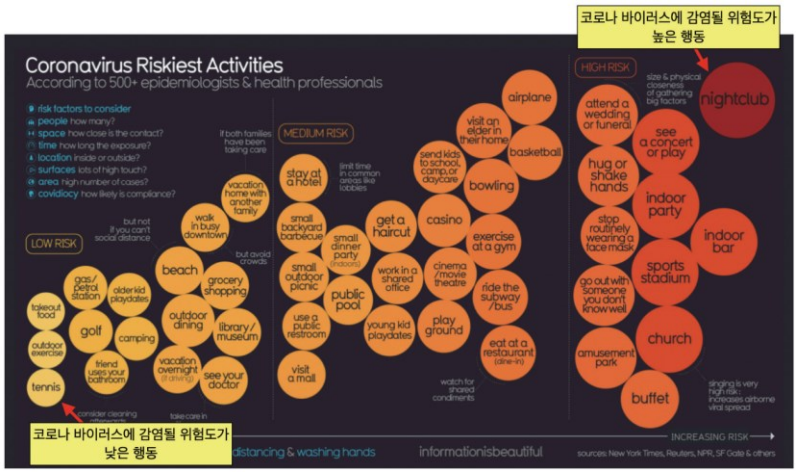
4.1 데이터 과학과 효과적인 시각화의 필요성



색상	개수
하늘색	5
회색	4
주황색	6
빨간색	11

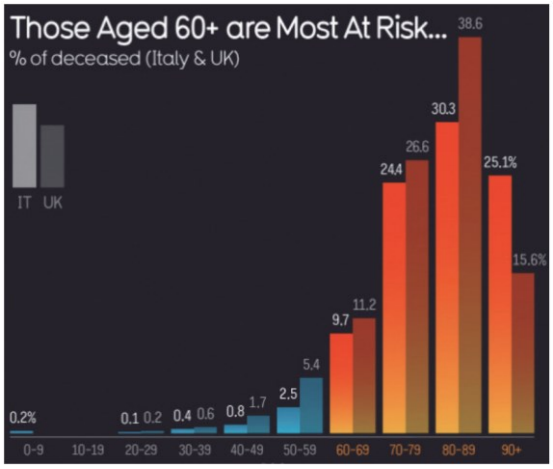
4.1 데이터 과학과 효과적인 시각화의 필요성

- informationisbeautiful.net이라는 웹사이트에서 제공하고 있는 멋진 시각화 사례의 일부이다.
- 이 웹사이트에서는 어떠한 행동이 코로나19 바이러스 감염에 취약한 행동인가를 다양한 색상과 크기를 가진 원으로 잘 표현하고 있다.



4.1 데이터 과학과 효과적인 시각화의 필요성

- COVID-19 감염질환은 나이가 많은 감염자의 치명률이 높지만 나이가 어린 감염자는 치명률이 낮은 편이다.
- 이처럼 데이터의 수집과 전처리 못지않게 **시각화를 통해서 그 의미를 잘 전달하는 것** 역시 데이터 과학자가 해야 할 중요한 일이다.



4.2 데이터 과학을 위한 시각화 도구 matplotlib

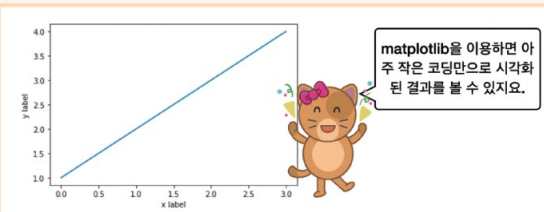
- 데이터를 시각화하기 위한 많은 라이브러리가 파이썬에 있지만 우리는 **matplotlib** 라이브러리를 사용할 것이다. **matplotlib**은 파이썬에서 가장 널리 사용되는 시각화 도구로 간단한 막대 그래프, 선 그래프, 산점도를 빠르게 그리는 용도로 적합하다.

- 그뿐만 아니라 고차원적인 데이터를 정밀하게 시각화하는 고급 기능도 가지고 있다. 우리는 **matplotlib**의 하위 모듈 중에서 **pyplot**이라는 서브(하위) 모듈을 주로 사용할 것이다. 이 서브 모듈에는 **시각화를 위한 핵심적인 함수들과 클래스들이 정의**되어 있다.

4.2 데이터 과학을 위한 시각화 도구 matplotlib

- 다음과 같은 간단한 예제 코드를 입력해서 레이블을 가진 이차원 평면에서 선 차트를 만들어보자.

```
import matplotlib as mpl
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4])
plt.ylabel('y label')
plt.xlabel('x label')
plt.show()     # 이 코드는 코랩에서는 없어도 됨
```



4.2 데이터 과학을 위한 시각화 도구 matplotlib

- **matplotlib**의 별칭은 **mpl**로, 이 모듈의 서브 모듈인 **pyplot**은 보통 **plt**라는 별칭으로 사용한다.

- **plt**의 **plot()** 함수는 리스트 자료값 [**1, 2, 3, 4**]를 **y** 값으로 삼아 화면에 그려주는 일을 한다 이때, **x** 값의 범위를 생략할 수 있는데 이 값은 디폴트로 [**0, 1, ... n**]으로 설정된다. **n**은 **y** 값의 개수에 의해 결정되므로, 여기서는 [**0, 1, 2, 3**]이 **x** 값의 범위가 된다. 따라서, 위의 코드에서 **plot([1, 2, 3, 4])** 부분은 다음 코드와 동일하다.

```
plt.plot([0, 1, 2, 3], [1, 2, 3, 4])
```

4.2 데이터 과학을 위한 시각화 도구 matplotlib

- **plt.xlabel()**과 **plt.ylabel()**은 각각 x 축과 y 축에 레이블을 적어주는 코드이며 각 축의 속성을 기록하면 된다.

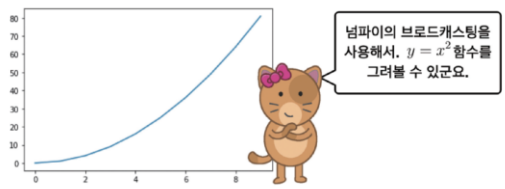
- 그리고, 가장 하단에 있는 **plt.show()**는 화면에 이 그래프를 그려주는 CHAPTER 04 데이터 시각화 도구 맷플롯립 107일을 하는데 구글의 코랩과 같은 주피터 노트북 기반의 환경에서는 **이 코드를 생략할 수 있다.**

- 하지만 **그 외의 경우에는 반드시 이 코드가 있어야만 화면에 출력이 이루어진다.** 이 책에서는 이 부분은 생략할 것이다.

4.2 데이터 과학을 위한 시각화 도구 matplotlib

- **plot()** 함수의 입력값으로 리스트를 사용할 수도 있으나, 다음과 같이 넘파이의 다차원 배열을 사용할 수도 있다. 이 경우 브로드캐스팅 기능을 통해서 쉽게 2차 함수를 그려볼 수 있다. 아래의 코드는 $y = x^2$함수를 시각화한 것으로 **x** 축의 구간은 **0**에서 **9** 사이의 값이다.

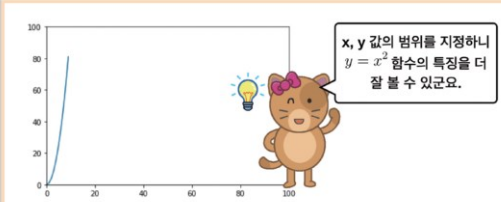
```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(10)    # 0에서 9 사이의 정수값을 생성함
plt.plot(x**2)       # x^2 함수를 그린다
```



4.2 데이터 과학을 위한 시각화 도구 matplotlib

- 이 코드에서 주의해야 할 내용은 **x** 축의 축척이 **0**에서 **9** 사이의 범위를 가지는데 비해, **y** 축은 **0**에서 **81**까지의 범위를 가진다는 점이다. 이 구간의 크기를 비슷하게 설정하려면 다음과 같이 **plt.axis()** 함수를 이용하여 **x** 축의 범위와 **y** 축의 범위를 모두 **0**에서 **100**으로 설정할 수 있다. 이 코드를 추가하여 실행하면 이전과는 달리 **x** 값이 증가할 경우 **y** 값이 매우 가파르게 증가하는 형태의 함수 모양을 제대로 볼 수 있다.

```
x = np.arange(10)            # 0에서 9 사이의 정수값을 생성함
plt.plot(x**2)               # x^2 함수를 그린다
plt.axis([0, 100, 0, 100])   # 범위를 지정함
```



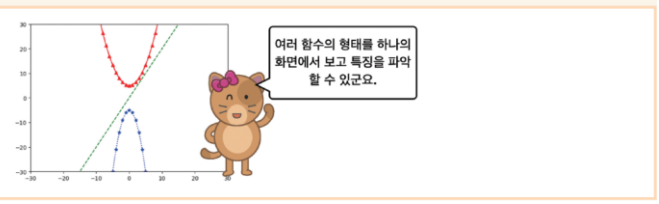
4.3 plot() 함수의 선 그리기 기능들을 알아보자

- 이번에는 **plot()** 함수를 수정하여 한 번의 호출만으로 $2x, (1/3)x^2 + 5, -x^2 - 5$의 세 함수를 그려보도록 하자.

- 이때 한 화면에 세 개의 함수가 모두 나타나야 하기 때문에 코드는 아래와 같이 다소 복잡하게 보일 수 있다.

```
import matplotlib.pyplot as plt
import numpy as np

x  = np.arange(-20, 20)  # -20에서 20 사이의 수를 1의 간격으로 생성
y1 = 2 * x               # 2*x를 원소로 가지는 y1 함수
y2 = (1/3) * x ** 2 + 5  # (1/3)*x**2 + 5를 원소로 가지는 y2 함수
y3 = -x ** 2 - 5         # -x**2 - 5를 원소로 가지는 y3 함수
# 빨강색 점선, 녹색 실선과 세모기호, 파랑색 별표와 점선으로 함수를 표현
plt.plot(x, y1, 'g--', x, y2, 'r^-', x, y3, 'b*:')
plt.axis([-30, 30, -30, 30])     # 그림을 그릴 영역을 지정함
```

- 이때, 다음과 같이 **plot()** 함수를 한 번만 호출하여 세 개의 함수를 그려보도록 하자.



4.3 plot() 함수의 선 그리기 기능들을 알아보자

- 첫 두 줄은 그림을 그리기 위한 **pyplot** 서브 모듈을 가져오는 작업이며, 마찬가지로 넘파이를 **np**라는 별칭으로 사용하는 작업이다. 또한 다음 코드는 넘파이에서 지정된 구간에 대하여 수의 시퀀스를 생성하여 다차원배열 x에 할당하는 코드이다.

```
x = np.arange(-20, 20) # -20에서 20 사이의 수를 1의 간격으로 생성
```

- 그 아래의 코드 역시 **y1, y2, y3** 함수를 정의하는 내용이다.

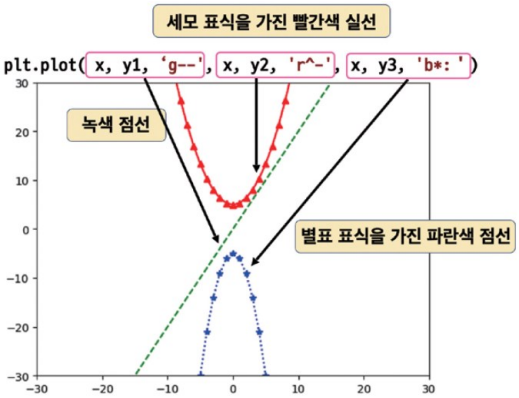
```
y1 = 2 * x                # 2*x를 원소로 가지는 y1 함수
y2 = (1/3) * x ** 2 + 5   # (1/3)*x**2 + 5를 원소로 가지는 y2 함수
y3 = -x ** 2 - 5          # -x**2 - 5를 원소로 가지는 y3 함수
```

4.3 plot() 함수의 선 그리기 기능들을 알아보자

- 다음의 코드는 y1, y2, y3 함수를 각각 다른 스타일로 그리는 코드이다.

```
plt.plot(x, y1, 'g--', x, y2, 'r^-', x, y3, 'b*:')
```

- 이 코드에서 **plot** 함수의 첫 인자와 두 번째 인자는 **x, y1**이 사용되었다.
- **x, y1**은 **x** 값과 **y** 값을 각각 나타내는데, 세 번째 인자인 **'g--'**인자를 통해서 이 함수를 어떻게 그릴지를 지정한다.
- 여기서는 녹색(green:g) 점선(--) 형태의 선으로 이 함수를 그릴 것을 기술하고 있다.



4.3 plot() 함수의 선 그리기 기능들을 알아보자

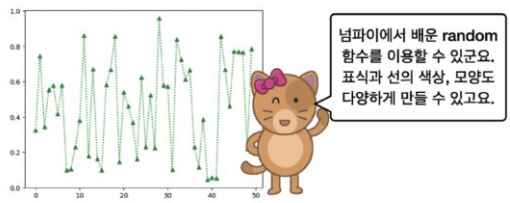
- 다음으로 나타나는 **x, y2** 역시 **y2** 함수를 그리라는 명령인데 **'r^-'** 인자를 통해서 이 함수를 어떻게 그릴지 지정한다.
- 여기서는 빨간색(**red:r**)의 실선(-)을 그리는데 해당 좌표에 세모(삼각형) 모양의 표식(^)을 나타낼 것을 기술한다.
- 다음으로 **y3** 함수는 'b*:'를 통해서 파란색(**blue:b**), 점선(:), 그리고 별표 표식(*)을 화면에 표시할 것을 나타내고 있다.
- 왼쪽의 표는 **matplotlib.org** 홈페이지에서 제공하는 다양한 **표식**marker의 일부이다.

marker	symbol	description
"."	•	point
",".	·	pixel
"o"	●	circle
"v"	▼	triangle_down
"^"	▲	triangle_up
"<"	◀	triangle_left
">"	▶	triangle_right
"1"	Y	tri_down
"2"	⅄	tri_up
"3"	≺	tri_left
"4"	≻	tri_right
"8"	●	octagon
"s"	■	square
"p"	⬟	pentagon
"P"	✚	plus (filled)
"*"	★	star

4.4 복잡한 선을 그리고 이미지로 저장하자

- **plot()** 함수는 매우 강력한 기능이 있는데 우선 다음과 같이 **0**에서 **50** 사이의 x 구간에 대하여, **0**에서 **1** 사이의 임의의 y 값을 넘파이의 **random()** 함수를 사용해서 생성하고 그려보자.

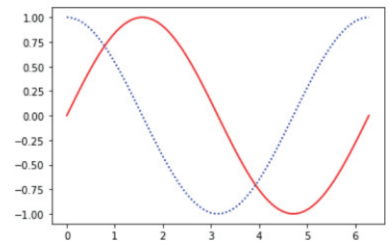
```
N = 50
x = np.arange(N)
y = np.random.random(size=N)

plt.plot(x, y, 'g^:') # 녹색(green) 점선, 삼각형의 표식 사용
```



4.4 복잡한 선을 그리고 이미지로 저장하자

- 난수 자료값을 그린 선분 대신, 넘파이의 sin(), cos()과 같은 주기함수를 사용하면 다음과 같이 나타낼 수 있다.

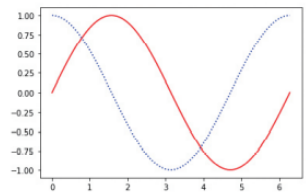
```
x = np.linspace(0, np.pi * 2, 100)      # 2*pi는 360도에 해당함

plt.plot(x, np.sin(x), 'r-')            # 사인함수는 빨강 실선으로
plt.plot(x, np.cos(x), 'b:')            # 코사인함수는 파랑 점선으로
```



4.4 복잡한 선을 그리고 이미지로 저장하자

- 난수 자료값을 그린 선분 대신, 넘파이의 sin(), cos()과 같은 주기함수를 사용하면 다음과 같이 나타낼 수 있다.
- 이와 같이 만들어진 그림은 **figure** 객체의 **savefig()** 메소드를 사용하여 저장할 수 있다.
- 이를 위해서는 반드시 다음과 같이 **plt.figure()**를 통해서 **figure** 객체를 생성하여야 한다.
- 여기에서는 **sin_cos_fig.png** 와 같이 **png** 파일 형식의 파일로 저장할 것을 지시하고 있다.

```
x = np.linspace(0, np.pi * 2, 100)      # 2*pi는 360도에 해당함

plt.plot(x, np.sin(x), 'r-')            # 사인함수는 빨강 실선으로
plt.plot(x, np.cos(x), 'b:')            # 코사인함수는 파랑 점선으로
```

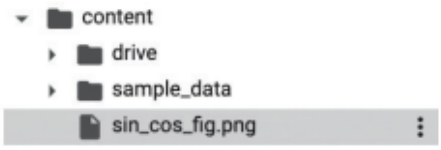


4.4 복잡한 선을 그리고 이미지로 저장하자

- **savefig()** 메소드로 저장 가능한 파일 형식은 **png, jpg, svg, tif, pdf**를 비롯한 **10**여 가지 이상의 다양한 파일 형식이 있다.
- 이 파일 형식은 **fig.canvas.get_supported_filetypes()** 명령으로 확인해 볼 수 있다.

```
x = np.linspace(0, np.pi * 2, 100)
fig = plt.figure()
plt.plot(x, np.sin(x), 'r-')
plt.plot(x, np.cos(x), 'b:')

fig.savefig('sin_cos_fig.png')
```

- **savefig()**로 저장된 파일은 구글의 코랩을 사용할 경우 다음과 같이 시스템의 **root** 디렉토리 아래 **content** 디렉토리에 저장되며 오른쪽 클릭을 통해서 여러분의 컴퓨터로 다운로드하는 것도 가능하다.

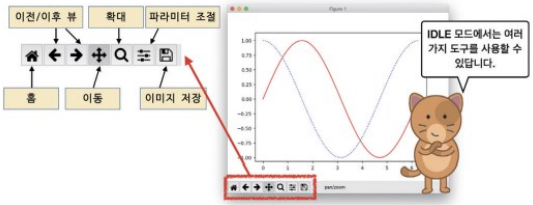


4.4 복잡한 선을 그리고 이미지로 저장하자

- 또한 다음과 같은 명령을 통해서 현재 노트북 화면에 이 그림을 불러올 수도 있다. 이 명령은 **IPython.display**라는 서브 모듈에 있는 **Image** 객체를 사용하여 파일의 내용을 화면에 출력하는 기능이다.

```
from IPython.display import Image
Image('sin_cos_fig.png')
```

- 만일 구글 코랩에서 실행하지 않고 코드를 **IDLE**에서 실행하게 되면 아래 그림과 같은 윈도우가 나타나는데 이 윈도우의 하단에 나타나는 아이콘을 이용하여 홈으로 이동하거나 이전, 이후 뷰로 이동, 단순 이동, 확대, 파라미터 조절, 이미지 저장 등의 기능을 이용할 수 있다.



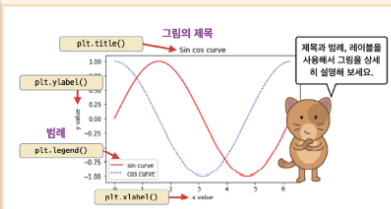
4.5 제목과 레이블, 스타일에 대해 알아보자

- 앞 절의 그림에 제목과 범례 등을 넣어서 그림을 좀 더 이해하기 쉽게 만드는 방법을 알아보 자.

- 그림의 제목은 **plt.title()** 함수로 만들 수 있으며, **plt.xlabel(), plt.ylabel()**로 x 축, y 축에 레이블링하는 것도 가능하다.

- 또한, **legend()** 함수를 통해서 범례를 추가하는 것도 가능하다.

```
x = np.linspace(0, np.pi * 2, 100)

plt.title('Sin cos curve')
plt.plot(x, np.sin(x), 'r-', label='sin curve')
plt.plot(x, np.cos(x), 'b:', label ='cos curve')
plt.xlabel('x value')
plt.ylabel('y value')
plt.legend()
```



4.5 제목과 레이블, 스타일에 대해 알아보자

- **legend()**를 사용하여 범례를 추가하기 위해서는 **plot()** 메소드의 **label** 인자에 이 메소드 의 설명글을 추가하는 작업이 필요하다.

- 만일 **label** 인자를 주지 않으면 **legend()** 메소드는 아무것도 출력하지 않을 것이다. 이 범례의 위치를 조절할 수도 있는데 **legend()**의 **loc** 인자 를 다음과 같이 설정하면 범례의 위치는 그림의 위쪽 오른편이 된다.

```
plt.legend(loc='upper right')  # 위쪽 오른편에 범례를 표시
```

- matplotlib에서 사용할 수 있는 범례의 위치는 다음과 같다.

```
'best', 'upper right', 'upper left', 'lower left', 'lower right', 'right', 'center
left', 'center right', 'lower center', 'upper center', 'center'
```

4.5 제목과 레이블, 스타일에 대해 알아보자

- **matplotlib**의 그림을 그릴 때는 하나의 스타일만이 아닌 다양한 스타일을 사용할 수 있다.

- 이를 테마 혹은 **스타일시트**stylesheet라고 하는데, 다음과 같은 코드를 추가하면 격자모양의 무늬가 나타나는 것을 볼 수 있다.

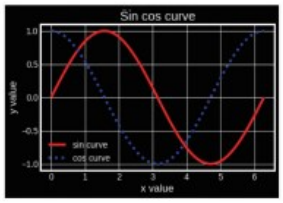
```
plt.style.use('seaborn-whitegrid')
```

4.5 제목과 레이블, 스타일에 대해 알아보자

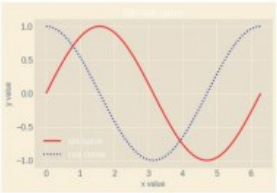
- **plt**의 **style** 객체는 **use()** 메소드를 가지는데 이 메소드는 미리 만들어져있는 그림판의 스타일을 지정할 수 있다.

- 예를 들어 'seaborn-whitegrid' 스타일을 지정할 경우 그림의 왼쪽과 같이 흰색 배경에 격자 모양의 패턴이 나타난다. 이처럼 미리 정의된 몇 가지 스타일과 그 스타일에 따른 출력 결과가 아래 그림에 있다.

'seaborn-whitegrid'



'dark_background'



'Solarize_Light2'

4.5 제목과 레이블, 스타일에 대해 알아보자

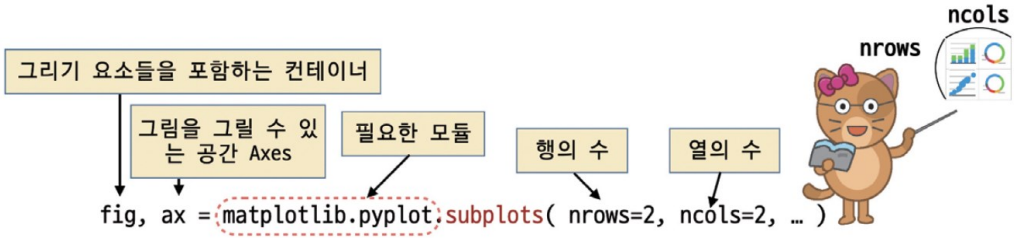
- 다음 코드는 현재 여러분이 사용하고 있는 **matplotlib**에서 사용 가능한 스타일들의 목록을
- 보여줄 것이며, 디폴트 스타일로 설정을 변경하고 싶다면 **'default'**를 입력한다.

```
print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', 'bmh', 'classic', 'dark_
background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn',
'seaborn-bright', 'seaborn-colorblind', 'seaborn-dark', 'seaborn-dark-
palette', 'seaborn-darkgrid', 'seaborn-deep', 'seaborn-muted', 'seaborn-
notebook', 'seaborn-paper', 'seaborn-pastel', 'seaborn-poster', 'seaborn-
talk', 'seaborn-ticks', 'seaborn-white', 'seaborn-whitegrid', 'tableau-
colorblind10']
```

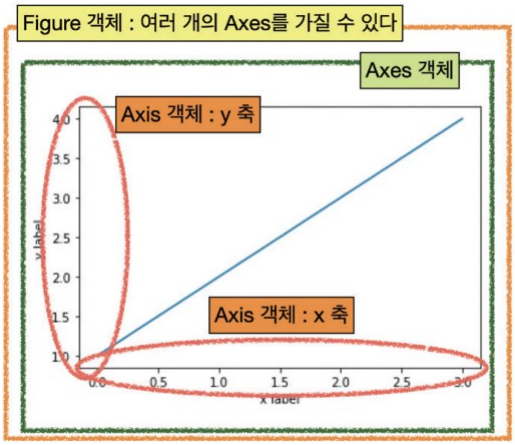
4.6 Figure, axes에 대하여 살펴보자

- matplotlib의 pyplot 서브 모듈의 plot() 함수를 사용해서 그림을 그리는 것은 간단한 그림을 빠르게 그리는데 편리하기는 하지만 다양한 종류의 그래프를 한 화면에 여러 개 그리기에는 부족하다.
- 이 기능을 사용하기 위해서는 pyplot 모듈의 subplots() 함수를 사용해야 한다.
- 이 함수는 다음과 같이 사용하며, 주로 필요한 행의 수와 열의 수를 입력한다.



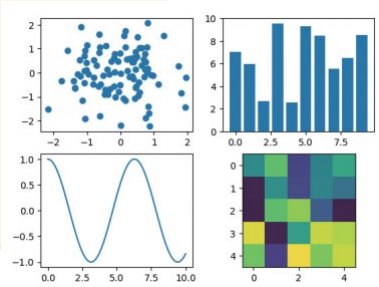
4.6 Figure, axes에 대하여 살펴보자

- subplots() 함수는 Figure 객체와 Axes 객체를 반환하는데 Figure 객체는 축과 그래픽, 텍스트 등의 **그리기 객체들을 포함하는 컨테이너 객체**이며, Axes 객체는 그림을 그릴 수 있는 테두리 상자 모양의 공간이다.
- ax는 다차원 배열 객체이므로 ax[0, 0]과 같은 인덱스를 사용하여 특정한 Axes 객체에 그림을 그릴 수 있다.
- 반면 Axis는 x, y축 객체이니 표기가 유사한 **Axes와 Axis를 혼동하지 않도록 주의**하도록 하자.



4.6 Figure, axes에 대하여 살펴보자

```
fig, ax = plt.subplots(2, 2)      # 2행 2열의 그림을 그리는 공간을 만든다

X = np.random.randn(100)          # 정규 분포를 가지는 데이터
Y = np.random.randn(100)          # 정규 분포를 가지는 데이터
ax[0, 0].scatter(X, Y)            # 산점도 그림
X = np.arange(10)                 # 0에서 9 사이의 연속값
Y = np.random.uniform(1, 10, 10) # 균일분포값 생성
ax[0, 1].bar(X, Y)                # 막대 차트 그림
X = np.linspace(0, 10, 100)
Y = np.cos(X)
ax[1, 0].plot(X, Y)               # 실선으로 함수를 그림
Z = np.random.uniform(0, 1, (5, 5))
ax[1, 1].imshow(Z)                # 분포를 2D 이미지로 그림
```



4.6 Figure, axes에 대하여 살펴보자

- 이 코드의 결과는 그림과 같이 산점도 그림, 막대 차트 그림, 실선 함수 그림, 이차원 이미지와 같이 각기 다른 스타일의 그림이 나타난다.

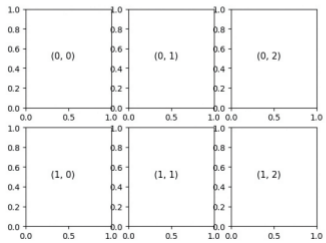
- 그림을 그리는 함수와 그 기능은 아래 표와 같다.

scatter()	산점도 그림을 그리는 기능으로 2차원 좌표에 흩뿌려진 점들의 좌표로 자료값을 나타냄.
bar()	막대 그래프를 그리는 기능, 막대의 모양은 수직 막대.
barh()	수평 막대 그래프를 그리는 기능.
pie()	파이 형태의 그림을 그리는 기능.
imshow()	이미지를 화면에 그리는 기능.
plot()	실선이나 점선으로 데이터를 나타냄.

4.7 subplot()의 고급기능

- 다음은 2행 3열의 격자 공간을 만들고 이 공간에 그리기 객체를 배치하는 방법이다.
- 우선 subplots() 메소드의 인자를 2, 3으로 두고 모두 6개의 그리드를 생성하였다.

```
fig, ax = plt.subplots(2, 3)

for i in range(2):      # 2행
   for j in range(3):   # 3열
       ax[i, j].text(0.3, 0.5, str((i,j)),fontsize = 11) # 텍스트만 표시
```



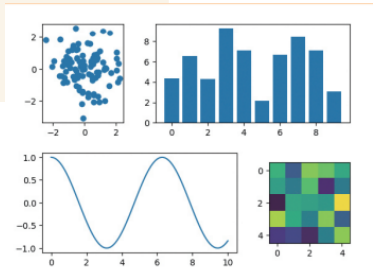
4.7 subplot()의 고급기능

- 아래의 코드에서 인자 2, 3은 행과 열의 수를 나타내며 wspace는 수평 여백을 표시하는데 이 값은 axis의 크기에 대한 상대적인 비율값이다.
- 여기서는 0.4로 설정하여 비교적 여유있는 너비 여백을 지정하였다. hspace 역시 수직 여백을 표시하는 비율값이다.
- 이 값을 **0으로 줄 때 격자 그림 사이의 간격은 사라진다는 점**에 유의하자.

```
fig, ax = plt.subplots(2, 3)

grid = plt.GridSpec(2, 3, wspace=0.4, hspace=0.3)
X = np.random.randn(100)  # 정규 분포를 가지는 데이터
Y = np.random.randn(100)  # 정규 분포를 가지는 데이터
plt.subplot(grid[0, 0]).scatter(X, Y)     # 산점도 그림
```

4.7 subplot()의 고급기능

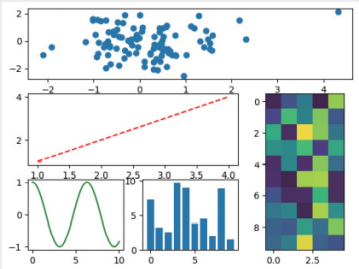
```
X = np.arange(10)                    # 0에서 9 사이의 연속값
Y = np.random.uniform(1, 10, 10)     # 균일분포값 생성
plt.subplot(grid[0, 1:]).bar(X, Y)   # 막대 차트 그림

X = np.linspace(0, 10, 100)
Y = np.cos(X)
plt.subplot(grid[1, :2]).plot(X, Y)  # 실선으로 함수를 그림

Z = np.random.uniform(0, 1, (5, 5))
plt.subplot(grid[1, 2]).imshow(Z)    # 분포를 2D 이미지로 그림
```



4.7 subplot()의 고급기능

도전문제 4.1(난이도 : 중)

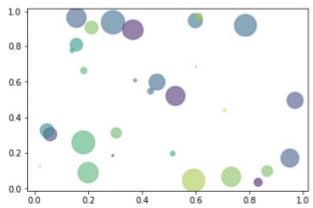
다음과 같은 그림을 subplots 함수와 GridSpec 클래스를 사용하여 화면에 그리도록 하여라. 이 화면을 구성하기 위해서는 5개의 격자 공간이 필요하며 산점도와 막대 그래프, 이미지 자료값은 임의의 값을 사용하도록 하여라.



4.8 자료값의 분포를 나타내는 산점도와 막대 그래프

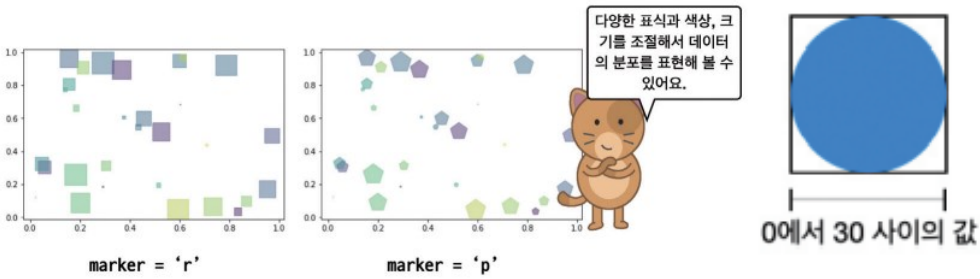
- 산점도는 두 변수의 상관관계를 직교 좌표계의 평면에 데이터를 점으로 표현하는 그래프로 scatter() 함수를 사용하면 쉽게 산점도를 그릴 수 있다.

```
N = 30                                  # 전체 점의 개수
x = np.random.rand(N)                   # 점들의 임의의 x 좌표
y = np.random.rand(N)                   # 점들의 임의의 y 좌표
colors = np.random.rand(N)              # 랜덤한 색상
area = (30 * np.random.rand(N))**2      # 반지름이 0에서 15 포인트 범위

plt.scatter(x, y, s=area, c=colors, alpha=0.5)
```



4.8 자료값의 분포를 나타내는 산점도와 막대 그래프

- **scatter()** 함수의 인자 중에서 s 키워드 인자는 점의 면적, c는 점의 색상, alpha는 투명도를 나타낸다.
- 점의 면적 s는 포인트 단위로 출력하는데 여기에서는 (30 * 랜덤값)의 거듭제곱을 사용한다.
- 넘파이의 랜덤값의 범위가 0에서 1 사이의 값이므로 이 범위는 0에서 30가 된다.

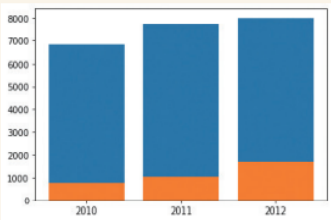


4.8 자료값의 분포를 나타내는 산점도와 막대 그래프

- **marker** 키워드 인자를 넣지 않으면 디폴트 표식은 원이 되는데, 이 표식을 그림과 같이 사각형(r:rectangle)과 오각형(p:pentagon) 등의 다양한 모양으로 변경할 수도 있다.

- 막대 그래프는 기본적으로 세로 직사각형 모양으로 데이터값을 나타낼 수 있다.

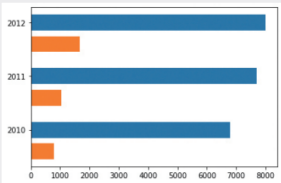
```
x = np.arange(3)
years = ['2010', '2011', '2012']
domestic = [6801, 7695, 8010]
foreign = [777, 1046, 1681]

plt.bar(x, domestic)
plt.bar(x, foreign)
plt.xticks(x, years)
```



4.8 자료값의 분포를 나타내는 산점도와 막대 그래프

도전문제 4.2(난이도 : 중)

barh()는 수평 막대 그림을 그리는 함수이다. 위의 제주도 관광객 방문 그래프 그림을 barh() 함수를 사용하여 다음과 같은 수평 막대 그림으로 그려보도록 하자.

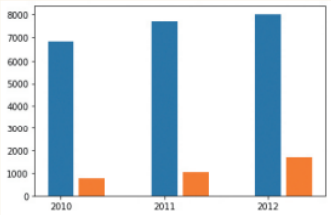


4.8 자료값의 분포를 나타내는 산점도와 막대 그래프

- 다소 보기에 불편한 이 자료값은 다음과 같이 **width** 키워드 인자를 통해 막대의 너비를 0.25로 줄 수 있다.

- 이 값의 의미는 원래 두께의 25%를 의미하며, x 값에 0.3을 더하여 x 축의 간격을 조절하는 것도 가능하다.

```
x = np.arange(3)
years = ['2010', '2011', '2012']
domestic = [6801, 7695, 8010]
foreign = [777, 1046, 1681]

plt.bar(x, domestic, width=0.25)
plt.bar(x + 0.3, foreign, width=0.25)
plt.xticks(x, years)
```

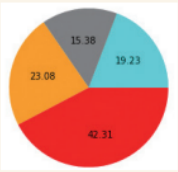


4.9 파이 차트와 히트맵 표현

- 이 장의 첫 그림에 나타난 다음과 같은 분포를 가지는 자료들이 있다고 가정하자.

- 이와 같은 **범주형 데이터**categorical data들이 가지는 전체적인 비율을 한눈에 보고자 할 때는 막대 그래프나 산점도 그래프보다 파이 형태의 그래프가 매우 적절할 것이다.

```
import matplotlib.pyplot as plt
import numpy as np

data = [5, 4, 6, 11]
clist = ['cyan', 'gray','orange', 'red']
plt.pie(data, autopct = '%.2f', colors=clist)
```



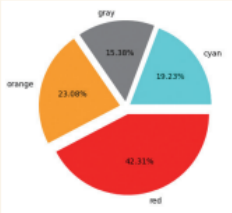
색상	개수
하늘색	5
회색	4
주황색	6
빨간색	11

4.9 파이 차트와 히트맵 표현

- **pie()** 함수의 인자인 **autopct**는 부채꼴 안에 표시될 숫자의 형식을 지정하며 **%.2f** 값은 소수점 아래 둘째 자리까지를 표시하라는 명령이다.

- 만일 %까지 표시하고자 할 경우 %.2f%% 와 같이 %를 두개 연속해서 입력해야만 한다. **colors** 키워드 인자는 각 섹션의 색상을 지정하는 역할을 하는데 이를 생략하면 디폴트 색상을 부여한다.

```
import matplotlib.pyplot as plt
import numpy as np

data = [5, 4, 6, 11]
clist = ['cyan', 'gray','orange', 'red']
explode = [.06, .07, .08, .09]
plt.pie(data, autopct = '%.2f%%',
        colors=clist, labels=clist, explode=explode)
```



4.9 파이 차트와 히트맵 표현

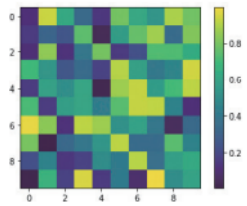
- **히트맵**heat map이란 열을 의미하는 **heat**와 지도를 의미하는 map을 결합한 단어로 이미지 위에 **열 분포 형태로 정보를 표현하는 방법**을 의미한다.

- 열 분포란 이 사례와 같이 실제 지표의 온도를 의미하기도 하지만 데이터의 값이 클 경우와 작은 경우에 대하여 서로 다른 색상을 사용하여 데이터의 분포를 쉽게 이해할 수 있는 유용한 기능이다.

4.9 파이 차트와 히트맵 표현

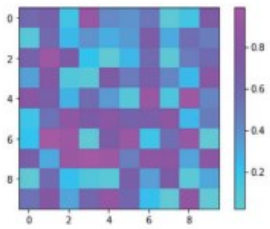
- 다음의 코드는 10×10 크기의 격자 공간에 0에서 1 사이의 임의의 수를 할당한 후 이 값들을 히트맵을 이용하여 그려본 것이다.
- 그림의 오른쪽에 있는 색상 막대를 살펴보면 이 색상의 범위가 0에서 1 사이임을 알 수 있다.

```
data = np.random.random((10, 10))  # 임의의 수를 가진 10×10 크기의 배열
plt.imshow(data)
plt.colorbar()
```

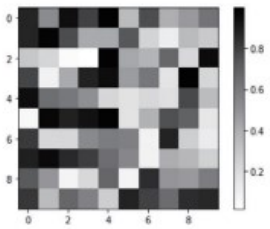


4.9 파이 차트와 히트맵 표현

- **imshow()** 함수 내의 키워드 인자로 **color map**의 약어인 **cmap**을 넣을 수 있는데, 이 키워드 인자에 다음과 같이 'cool', 'Greys' 등과 같은 색상맵 타입을 설정하여 다양한 색상을 가진 히트맵을 얻을 수 있다.



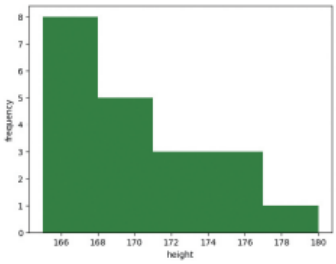
cmap = 'cool'



cmap = 'Greys'

4.10 히스토그램

- **히스토그램**histogram은 주어진 자료를 몇 개의 구간으로 나누고 각 구간의 **도수**frequency를 조사하여 나타낸 막대 그래프이다. 이 그림의 히스토그램의 가로 축은 구간, 세로 축은 도수를 나타낸다.



- 히스토그램의 예제로 동민이네 반 30명 학생의 키를 조사하여 그 분포를 지정된 구간에 나타내고자 한다. 다음의 heights 넘파이 배열 자료는 다음과 같이 학생들의 키를 모은 자료이다.

```
heights = np.array([175, 165, 164, 164, 171, 165, 160, 169, 164, 159, 163, 167,
163, 172, 159, 160, 156, 162, 166, 162, 158, 167, 160, 161, 156, 172, 168, 165, 165,
177])
```

4.10 히스토그램

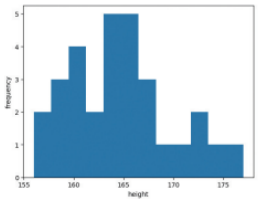
- 이 자료들을 사용하여 히스토그램을 만들어주려면 데이터의 값을 동일한 구간으로 나눠야 한다. 데이터가 들어가는 통을 **빈**bin이라고 한다.

- 다음 코드는 **heights** 자료값을 6개의 빈에 넣어서 히스토그램으로 시각화하는 코드이다. **plt.hist()** 함수에 **heights** 인자와 **bins** 키워드 인자를 넣는 것만으로 손쉽게 히스토그램을 그려볼 수 있다.

```
heights = np.array([175, 165, 164, 164, 171, 165, 160, 169, 164, 159, 163,
167, 163, 172, 159, 160, 156, 162, 166, 162, 158, 167, 160, 161, 156, 172,
168, 165, 165, 177])
plt.hist(heights, bins=6)      # 히스토그램 bins는 구간의 수
plt.xlabel("height")
plt.ylabel("frequency")
```

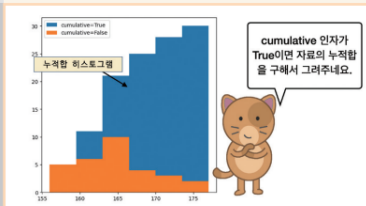
4.10 히스토그램

- 이 히스토그램의 bins 값을 크게 하면 할수록 그림과 같이 많은 구간에 자료값이 들어가는 것을 볼 수 있는데, 오른쪽 그림과 같이 bins가 12로 너무 크면 오히려 자료값들의 분포를 살펴보는데 방해가 될 수 있다.



- 이 코드는 아래 그림의 결과와 같이 청색 히스토그램에 누적된 자료값의 합을 출력하고 있는데, 데이터의 개수가 30이므로 가장 오른쪽의 히스토그램 높이가 30인 것도 확인해 볼 수 있다.

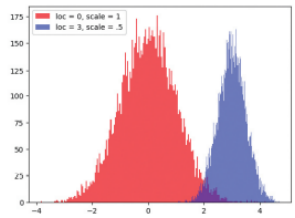
```
plt.hist(heights, bins=6, label='cumulative=True', cumulative=True)
plt.hist(heights, bins=6, label='cumulative=False', cumulative=False)
plt.legend(loc='upper left')
```



4.11 히스토그램을 이용한 정규 분포 함수와 확률 밀도 함수 그리기

- **정규 분포**normal distribution란 연속 확률 분포의 하나이며 평균이 0이고 분산이 1인 정규 분포를 표준 정규 분포라 한다.

```
f1 = np.random.normal(loc=0, scale=1, size=10000)
f2 = np.random.normal(loc=3, scale=.5, size=10000)

plt.hist(f1, bins=200, color='red', alpha=.7,
         label='loc = 0, scale = 1')
plt.hist(f2, bins=200, color='blue', alpha=.5,
         label='loc = 3, scale = .5')
```



4.11 히스토그램을 이용한 정규 분포 함수와 확률 밀도 함수 그리기

- 다음 코드를 살펴보면 1,000개의 데이터를 생성하여 히스토그램으로 그리는 과정은 앞의 내용과 동일하다.

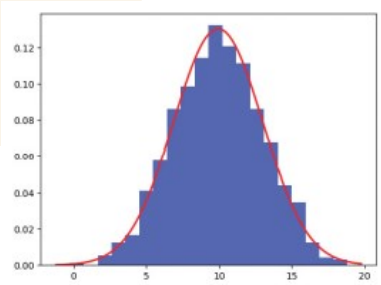
- 이때 hist() 함수의 density 인자가 True로 설정되어 있음을 눈여겨 보도록 하자. 이 값이 True로 설정될 경우 히스토그램을 그리면서 동시에 **확률 밀도**probability density 값을 반환한다.

- 따라서 이 데이터를 바탕으로 norm.fit() 함수를 사용할 경우 평균(mu)과 표준편차(std)를 얻을 수 있다.

- 이 평균과 표준편차를 바탕으로 **확률 밀도 함수**probability density function를 빨간색으로 그리게 되면 아래 그림과 같이 히스토그램과 확률 밀도 함수를 한 화면에 그려볼 수 있다.

4.11 히스토그램을 이용한 정규 분포 함수와 확률 밀도 함수 그리기

```
from scipy.stats import norm

# 평균이 10.0이고 분산이 3인 1,000개의 데이터를 생성
data = norm.rvs(10.0, 3, size=1000)
# 히스토그램을 그림
plt.hist(data, bins=20, density=True, alpha=0.6, color='b')

# 정규 분포를 데이터에 피팅함
mu, std = norm.fit(data)
# scipy의 pdf는 평균, 표준편차로부터 확률 밀도 함수를 생성한다
xmin, xmax = plt.xlim()  # x의 최소, 최대값을 pyplot으로부터 구함
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, mu, std)
plt.plot(x, p, 'r', linewidth=2)
```



4.12 상자 수염 그리기

- **상자 수염 그림**box-and-whisker plot은 수치 데이터를 표현하는 그래프의 한 종류이다. 이 그림은 흔히 **상자 차트**box plot라고도 한다.

- 이 그래프는 **최대, 최소, 중간값과 사분위 수 등의 통계량을 하나의 상자 그림으로 가시화**할 수 있는 방법이다.

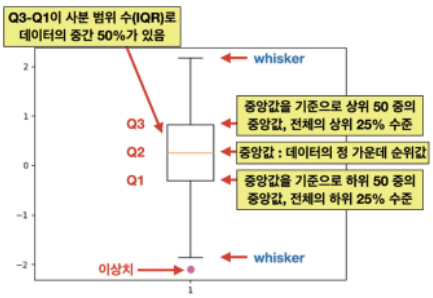
```
# 넘파이의 난수 모듈에 있는 정규 분포에 따르는 난수 100개를 생성
rand_data = np.random.randn(100)

plt.boxplot(rand_data)
```

4.12 상자 수염 그리기

- 이 그림에서 나타난 상자 모양 직사각형은 주황색으로 나타난 **중앙값median** Q2를 중심으로 상위 25%(Q3로 나타남)와 하위 25%(Q1으로 나타남)의 값이 모여 있는 구간을 표시한다.

- 이 그림에서 상자의 상단인 Q3에서 상자의 하단인 Q1의 값을 뺀 Q3-Q1을 **사분 범위** 혹은 **IQR**inter-quartile range이라고 한다.

- 그리고 상자 아래위로 직선이 그어져 있고 이를 위와 아래에서 막는 선이 있는데 이것을 **위스커**whisker 혹은 수염이라고 한다.



4.12 상자 수염 그리기

- 상단 수염과 하단 수염의 범위는 다음과 같다.

– 상단 수염 : Q2 + 1.5 * IQR 값보다 작은 데이터 중 가장 큰 값 – 하단 수염 : Q2 – 1.5 * IQR 값보다 큰 데이터 중 가장 작은 값

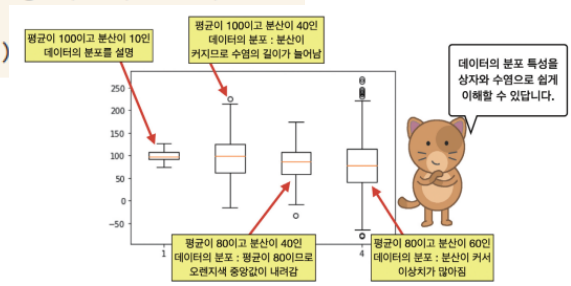
- 이 데이터 분포의 아래, 위 1.5배 이내의 자료는 수염으로 표시하고 수염의 끝에 막대를 그어 **이상치**outlier 자료 값의 경계를 표시하는 데 유용하게 사용할 수 있다.

- 즉 아래쪽 위스커에서 위쪽 위스커까지의 범위가 대부분의 데이터가 분포하는 범위라고 볼 수도 있다.

4.12 상자 수염 그리기

- 이제 다음과 같이 네 종류의 데이터가 준비되어 있다고 할 때, 이들을 각각 상자 차트로 그려 보도록 하자.

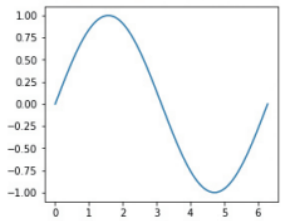
```
np.random.seed(85)
data1 = np.random.normal(100, 10, 200) # 평균이 1000이고 분산이 10
data2 = np.random.normal(100, 40, 200) # 평균이 1000이고 분산이 40
data3 = np.random.normal(80, 40, 200)  # 평균이 800이고 분산이 40
data4 = np.random.normal(80, 60, 200)  # 평균이 800이고 분산이 60

plt.boxplot([data1, data2, data3, data4])
```



4.13 그래프의 크기와 그리드 그리기

- **matplotlob**에는 화면에 나타날 그래프의 크기를 조절하는 기능도 있는데 **plt.figure()** 함수의 키워드 인자로 **figsize**를 줄 수 있다.

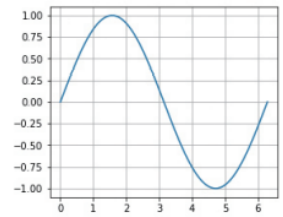
```
X = np.linspace(0, 2*np.pi, 200)
Y = np.sin(X)
plt.figure(figsize=(4.2, 3.6))
plt.plot(X, Y)
```



4.13 그래프의 크기와 그리드 그리기

- **matplotlib**의 **pyplot** 모듈에 있는 **grid()** 함수는 다음과 같이 그래프의 내부에 격자를 그려서 그래프의 모양을 좀 더 정확하게 이해하는 데 도움을 준다.

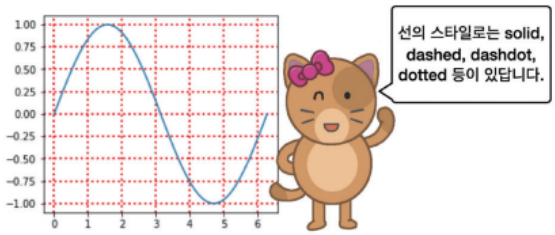
```
X = np.linspace(0, 2*np.pi, 200)
Y = np.sin(X)
plt.figure(figsize=(4.2, 3.6))
plt.plot(X, Y)
plt.grid()
```



4.13 그래프의 크기와 그리드 그리기

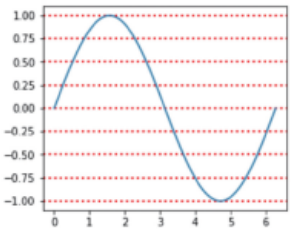
- 이 코드에서 **grid()** 함수의 내부에 키워드 인자로 **color, linestyle, linewidth** 등을 사용하여 격자의 색상, 선의 형태, 선의 굵기를 변경할 수도 있다.

```
# plt.grid() 대신 다음과 같은 키워드 인자를 지정하여 호출
plt.grid(color='r', linestyle='dotted', linewidth=2)
```

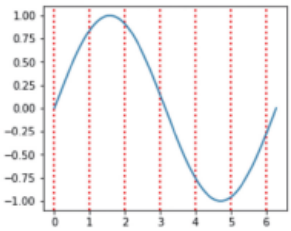


4.13 그래프의 크기와 그리드 그리기

- 이 밖에도 axis='y'나 axis='x' 인자를 사용할 경우 수평선, 수직선 격자 선분을 그릴 수도 있다.



axis='y' 인자를 사용한 경우



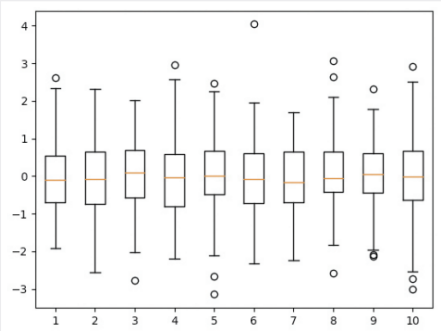
axis='x' 인자를 사용한 경우

4.13 그래프의 크기와 그리드 그리기

도전문제 4.3(난이도 : 상)

오른쪽 그림과 같이 100개의 난수를 가진 데이터 10종을 만들어 각각의 데이터 범위를 가시화하는 코드를 작성해 보라. 난수를 생성하는 일은 넘파이의 random 모듈이 가지고 있는 randn(d0, d1)을 사용하면 될 것이다.

이때 생성되는 데이터는 넘파이 배열이므로 차원을 어떻게 결정해야 하는지 유의해서 숫자 d0와 d1을 결정해야 할 것이다. 결과는 생성된 난수에 따라 오른쪽 그림과 조금 다를 것이다.



Questions?